ShopEase: Object-Oriented Architecture for a Scalable E-Commerce Platform

Introduction

Designing a system like ShopEase means balancing several pressures at once. The platform needs to cope with a growing number of users and products without falling over, it needs to stay maintainable as new features get bolted on over time, and it needs to protect customer data and payment details from day one rather than as an afterthought. A layered object oriented architecture is a sensible way to manage that tension, since it separates concerns cleanly enough that a change to how products are stored, for instance, does not ripple through into how the website renders a product page. Fowler (2002) describes this kind of separation as one of the more durable ideas in enterprise application design, precisely because each layer can evolve, be tested, and be scaled independently of the others.

Layered Architecture Overview

ShopEase is structured into three layers. The Presentation Layer handles anything the user directly sees or interacts with, whether that is a web browser hitting the storefront or a mobile app calling the same backend through an API, and it is also responsible for input validation and session management, so malformed requests and unauthenticated activity are caught before they ever reach the layers underneath. The Business Logic Layer is where the actual rules of the system live, authentication, product search, pricing, order processing, and it is deliberately kept independent of how requests arrive or how data is stored. The Data Access Layer manages persistence, translating objects in the business layer into rows in a database and back again, while also handling caching and file storage. Keeping these layers strictly separated means the Presentation Layer never talks to the database directly, and the Business Logic Layer never has to know whether a request came from a browser or a phone.

Presentation Layer

The Presentation Layer is kept intentionally thin. Its job is to validate input, manage the user's session, hand the request to the correct service in the Business Logic Layer, and format whatever comes back, not to contain business rules of its own. Where the web and mobile clients need to render the same underlying data differently, a Strategy Pattern is used, so the controller selects a rendering strategy at runtime rather than branching on client type throughout the code (Gamma et al., 1994).

python

```
class RenderStrategy:
    def render(self, product):
        raise NotImplementedError


class WebRenderStrategy(RenderStrategy):
    def render(self, product):
        return {"html": f"<div>{product.name} - £{product.price}</div>"}


class MobileRenderStrategy(RenderStrategy):
    def render(self, product):
        return {"json": {"name": product.name, "price": product.price}}


class ProductController:
    def __init__(self, render_strategy):
        self.render_strategy = render_strategy

    def show_product(self, product):
        return self.render_strategy.render(product)
```

This also means that if ShopEase later wanted to add a third channel, a partner integration for instance, it would only need a new rendering strategy rather than a rebuild of the order processing rules underneath it.

Business Logic Layer

This is where most of the object oriented design work happens, and it is organised into three cooperating modules.

User Management Module

The User class encapsulates a customer's identity and keeps sensitive fields like the password hash private, exposing only the behaviour needed to authenticate or update a profile rather than raw access to the data itself. Encapsulation matters here specifically because it stops other parts of the system reaching in and manipulating a password field directly, which is exactly the kind of accidental coupling that turns into a security problem later.

python

```
class User:
    def __init__(self, user_id, email, password_hash):
        self._user_id = user_id
        self._email = email
        self._password_hash = password_hash

    def verify_password(self, provided_password, hasher):
        return hasher.verify(provided_password, self._password_hash)

    def get_id(self):
        return self._user_id
```

Product Catalog Module

The product hierarchy is where inheritance and polymorphism earn their place. A base Product class holds the attributes every product shares, name, price, stock level, and subclasses like DigitalProduct and PhysicalProduct override behaviour that genuinely differs between them, such as how shipping cost is calculated.

python

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def calculate_shipping(self):
        return 0.0


class PhysicalProduct(Product):
    def __init__(self, name, price, stock, weight_kg):
        super().__init__(name, price, stock)
        self.weight_kg = weight_kg

    def calculate_shipping(self):
        return self.weight_kg * 1.5


class DigitalProduct(Product):
    def calculate_shipping(self):
        return 0.0
```

Because the order processing code only ever calls calculate_shipping() on whatever Product object it is holding, it never needs an if statement checking the product type. Nguyen Quan (2022) makes a similar point about polymorphism removing exactly this kind of conditional branching, letting each subclass carry its own behaviour rather than centralising it in one place that has to know about every variant.

Order Processing Module

The Order class coordinates a list of products, calculates a total, and triggers the payment and notification steps. Deciding which concrete payment processor to use is a creation concern, and it is handled separately from how that processor is later used. A Factory Method decides which processor class to instantiate based on the payment type requested, keeping that branching logic in one place rather than scattered across the codebase (Gamma et al., 1994).

python

```
class PaymentProcessor:
```

```
    def process(self, amount):
        raise NotImplementedError


class CardPaymentProcessor(PaymentProcessor):
    def process(self, amount):
        print(f"Processing card payment of £{amount}")


class PayPalPaymentProcessor(PaymentProcessor):
    def process(self, amount):
        print(f"Processing PayPal payment of £{amount}")


class PaymentProcessorFactory:
    @staticmethod
    def create(payment_type):
        if payment_type == "card":
            return CardPaymentProcessor()
        if payment_type == "paypal":
            return PayPalPaymentProcessor()
        raise ValueError(f"Unsupported payment type: {payment_type}")
```

The processor the factory produces is then handed into OrderService through dependency injection, so the two patterns work together rather than replacing one another. Creation is decided by the factory, use is decided by whatever OrderService does with the object once it has it, and OrderService itself depends only on abstractions for payment processing and notifications rather than concrete implementations, which is what makes the extensibility requirement achievable without rewriting this class every time a new payment provider or notification channel is added.

Data Access Layer and Database Design

The Data Access Layer combines two closely related patterns. The Repository Pattern sits behind the Business Logic Layer and offers a higher level, object oriented view of persistence, a UserRepository that returns and saves User objects without the caller knowing what database sits underneath it. Data Access Objects (DAOs) sit a level lower and handle the narrower, table specific work of actually querying and mapping rows. Caching is layered on top of the repository so that frequently requested data, product listings especially, can be served from memory rather than hitting the database on every request, and file storage handles anything not suited to a relational table, such as product images.

python

```
class ProductDao:
    def __init__(self, connection):
        self.connection = connection

    def find_by_id(self, product_id):
        return self.connection.query(
```

```
            "SELECT * FROM products WHERE id = %s", (product_id,)
        )


class CachedProductRepository:
    def __init__(self, dao, cache):
        self.dao = dao
        self.cache = cache

    def get_by_id(self, product_id):
        cached = self.cache.get(product_id)
        if cached:
            return cached
        product = self.dao.find_by_id(product_id)
        self.cache.set(product_id, product)
        return product


class UserRepository:
    def get_by_id(self, user_id):
        raise NotImplementedError

    def save(self, user):
        raise NotImplementedError


class SqlUserRepository(UserRepository):
    def __init__(self, connection):
        self.connection = connection

    def get_by_id(self, user_id):
        row = self.connection.query(
            "SELECT * FROM users WHERE id = %s", (user_id,)
        )
        return User(row["id"], row["email"], row["password_hash"])

    def save(self, user):
        self.connection.execute(
            "UPDATE users SET email = %s WHERE id = %s",
            (user.get_id(),),
        )
```

For the underlying schema, a relational design fits ShopEase reasonably well given how structured the relationships between users, products and orders are. A Users table holds identity and authentication data, a Products table holds catalogue information with a separate Inventory table tracking stock levels so that concurrent purchases can be reconciled safely, an Orders table references a user and holds order level status, and an OrderItems table links orders to products with quantity and price at time of purchase, since product prices can change after an order is placed. Keeping price at time of purchase in OrderItems rather than only referencing the current product price avoids a subtle but common bug where historical orders silently change value if a product is later repriced. Product images and other unstructured assets are kept out of the relational schema entirely and handled through the file storage responsibility of this layer, with the database holding only a reference path to the stored file.

Security Practices

Authentication is the most immediate security concern for a system handling customer accounts and payment details. Passwords should never be stored as plain text or even reversibly encrypted, and instead should go through a slow, salted hashing algorithm designed specifically for password storage, as recommended in the NIST digital identity guidelines (NIST, 2020). The verify_password method shown earlier deliberately delegates to a hasher object rather than comparing strings directly, which also means the hashing algorithm itself can be swapped out later without changing the User class. Session management in the Presentation Layer works alongside this, since a valid session token is what proves a request belongs to an already authenticated user without requiring credentials to be resent on every call. Beyond that, OWASP (2021) sets out a wider set of practices relevant here, including protecting against injection attacks in the Data Access Layer by using parameterised queries rather than string concatenation, which the DAO and repository examples above already follow, and enforcing HTTPS across the Presentation Layer so credentials and session tokens are never sent in the clear.

Scalability and Extensibility

Dependency Injection

Rather than an Order or a User service constructing its own repository or payment processor internally, those dependencies are passed in from outside, typically at application startup, with the payment processor itself coming from the factory described earlier. This is what allows a test to substitute a fake repository for a real database connection, and it is also what allows ShopEase to swap in a different implementation, a caching repository or a different payment provider, without editing the class that depends on it.

python

```
class OrderService:
  def __init__(self, order_repository, payment_processor, notifier):
    self.order_repository = order_repository
    self.payment_processor = payment_processor
    self.notifier = notifier

  def place_order(self, order):
    self.payment_processor.process(order.get_total())
    self.order_repository.save(order)
    self.notifier.notify_order_placed(order)
```

Observer Pattern for Notifications

Order status changes need to trigger several independent reactions, an email confirmation, an SMS update, perhaps later a warehouse system being told to begin fulfilment, and the Order class itself should not need to know about any of

them individually. Gamma et al. (1994) describe the Observer pattern as exactly this kind of one to many dependency, where a subject notifies a list of observers without needing to know what any particular observer does with that notification. Applied here, the OrderService holds a list of notifier objects and calls a shared interface on each of them when an order's status changes, so adding a new notification channel later means writing one new observer class rather than modifying OrderService at all.

python

```python
class OrderObserver:
    def notify_order_placed(self, order):
        raise NotImplementedError


class EmailNotifier(OrderObserver):
    def notify_order_placed(self, order):
        print(f"Emailing confirmation for order {order.id}")


class SmsNotifier(OrderObserver):
    def notify_order_placed(self, order):
        print(f"Texting confirmation for order {order.id}")
```

Conclusion

The layered structure gives ShopEase a clear separation between how customers interact with the system, how the business rules around products and orders are enforced, and how data is actually stored, which addresses the modularity requirement directly. Encapsulation, inheritance and polymorphism inside the Business Logic Layer keep each module focused on its own responsibility, the Strategy Pattern lets the Presentation Layer render the same data differently per client without duplicating logic, the Factory Method keeps payment processor creation in one place, dependency injection keeps modules loosely coupled enough to test and extend, and the Observer pattern means new notification channels or downstream integrations can be added without touching existing, already tested code. None of this makes the system automatically secure or scalable on its own, but it does mean that scaling a database, swapping a payment provider, or hardening authentication later are each contained changes rather than system wide rewrites.

References

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.

McDonald, J.C. (2023) *Dead Simple Python: Idiomatic Python for the Impatient Programmer*. 1st edn. New York: No Starch Press.

Nguyen Quan (2022) *Advanced Python Programming*. 2nd edn. Birmingham: Packt Publishing.

NIST (2020) *Digital Identity Guidelines: Authentication and Lifecycle Management* (SP 800-63B). Gaithersburg: National Institute of Standards and Technology.

OWASP (2021) *OWASP Top 10:2021*. Available at: https://owasp.org/Top10/2021/ (Accessed: 16 September 2026).